



**PES**

**UNIVERSITY**

CELEBRATING 50 YEARS

# PROGRAMMING WITH PYTHON

---

**Lekha A**

Computer Applications

# PROGRAMMING WITH PYTHON

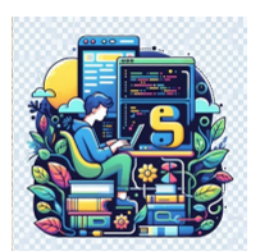
---

## Object-Oriented Programming (OOP) and Advanced Concepts

### Polymorphism

**Lekha A**

Computer Applications

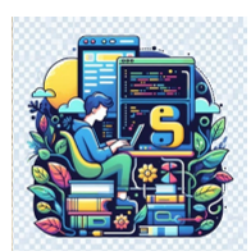


# PROGRAMMING WITH PYTHON

## Polymorphism

---

- The literal meaning of polymorphism is the condition of occurrence in different forms.
- It refers to the use of a single type entity (method, operator or object) to represent different types in different scenarios.



# PROGRAMMING WITH PYTHON

## Operator Polymorphism

```
a=10;b=5
```

```
print(a+b)
```

15

```
a="Hello"; b="Polymorphism"
```

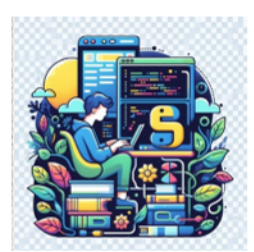
```
print(a+b)
```

HelloPolymorphism

```
a=[10,20,30];b=[30,40]
```

```
print(a+b)
```

[10, 20, 30, 30, 40]



# PROGRAMMING WITH PYTHON

## Function Polymorphism

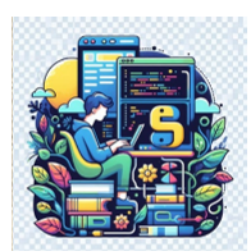
- *##Function Polymorphism*

```
print(len("Polymorphism"))
```

12

```
print(len([10,20,30,40]))
```

4



# PROGRAMMING WITH PYTHON

## Class Polymorphism

```
class Cat:
    def __init__(self, name, age):
        self.name = name
        self.age = age

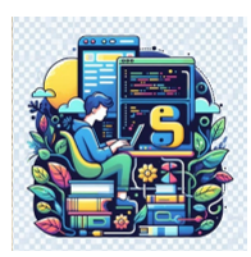
    def info(self):
        print(f"I am a cat. My name is {self.name}. I am {self.age} years old.")

    def make_sound(self):
        print("Meow")
```

```
class Dog:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def info(self):
        print(f"I am a dog. My name is {self.name}. I am {self.age} years old.")

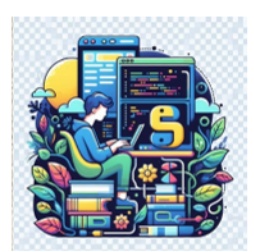
    def make_sound(self):
        print("Bark")
```



# PROGRAMMING WITH PYTHON

## Class Polymorphism

- Two classes Cat and Dog are created that share a similar structure and have the same method names `info()` and `make_sound()`.

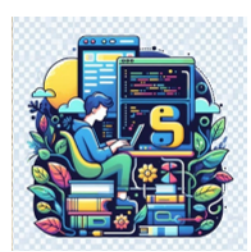


# PROGRAMMING WITH PYTHON

## Polymorphism and Inheritance

- Polymorphism is the ability to treat a class differently, depending on which subclass is implemented.
- It is the ability of objects to take on multiple forms or the ability of different classes to be treated as instances of the same class through a common interface.
- This is achieved through Method overriding.

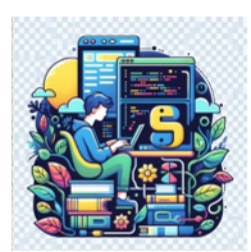




# PROGRAMMING WITH PYTHON

## Method Overriding

- Method overriding involves defining a method in a child class that has the same name and parameters as a method in its parent class.
- When the method is called on an instance of the child class, the child class's implementation of the method is executed instead of the parent class's implementation.

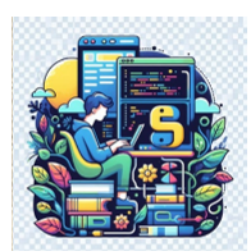


# PROGRAMMING WITH PYTHON

## Method Overriding

```
class Animal:  
    def make_sound(self):  
        print("Making a generic animal sound...")
```

- Create child classes like Dog and Cat that inherit from the Animal class and override the make\_sound() method to produce their specific sounds



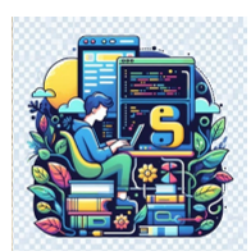
# PROGRAMMING WITH PYTHON

## Method Overriding

```
class Dog(Animal):  
    def make_sound(self):  
        print("Woof!")
```

```
class Cat(Animal):  
    def make_sound(self):  
        print("Meow!")
```

- When instances of Dog and Cat are created and make\_sound() method is called, the overridden methods in the respective child classes will be executed.



# PROGRAMMING WITH PYTHON

## Method Overriding

- `dog = Dog()`

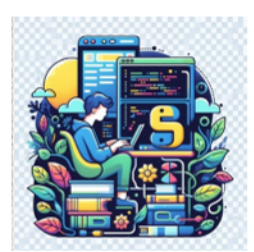
```
dog.make_sound()
```

Woof!

```
cat = Cat()
```

```
cat.make_sound()
```

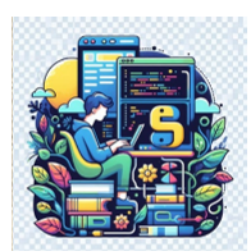
Meow!



# PROGRAMMING WITH PYTHON

## Duck Typing

- Duck Typing is a term commonly related to dynamically typed programming languages and polymorphism.
- The idea behind this principle is that the code itself does not care about whether an object is a duck, but instead it does only care about whether it quacks.
  - **If it walks like a duck, and it quacks like a duck, then it must be a duck.**

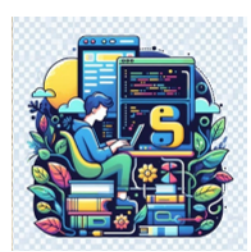


# PROGRAMMING WITH PYTHON

## Duck Typing

---

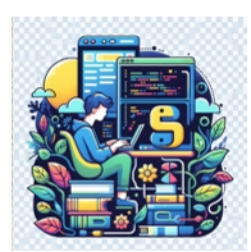
- It is important to observe that we are not checking internally if the two objects are the same, but rather using known external behavior to match the two objects.



# PROGRAMMING WITH PYTHON

## Duck Typing

- Define three classes representing different animals, a duck, a goose and a cat.
- They all produce different sounds.
- The Duck and Goose quack while the Cat makes a meow sound.

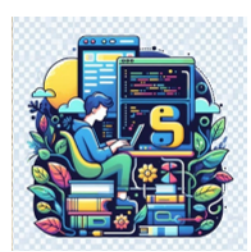


# PROGRAMMING WITH PYTHON

## Duck Typing

- Define a function `quack()` that takes in an animal and prints the description with the sound of a quack.
- It is obvious that an animal with no method of `quack()` will give an error when passed through the function, indicating a wrong type of variable( indicating it failed the duck test).





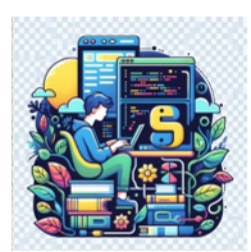
# PROGRAMMING WITH PYTHON

## Duck Typing

- ```
class Duck:
    def quack(self):
        print("I am a duck and I quack.")

class Goose:
    def quack(self):
        print("I am a goose and I quack.")

class Cat:
    def meow(self):
        print("I am a dog and I meow.")
```



# PROGRAMMING WITH PYTHON

## Duck Typing

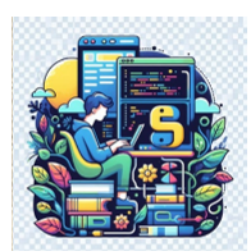
- ```
# Define the method  
def quack(animal):  
    animal.quack()
```

```
quack(Duck())
```

I am a duck and I quack.

```
quack(Goose())
```

I am a goose and I quack.



# PROGRAMMING WITH PYTHON

## Duck Typing

- `quack(Cat())`

-----  
**AttributeError**

Traceback (most recent call last)

Cell In[62], line 1

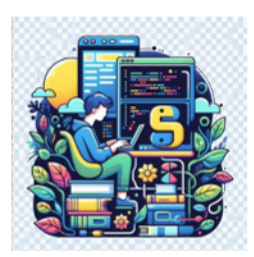
----> 1 `quack(Cat())`

Cell In[59], line 3, in `quack(animal)`

2 `def quack(animal):`

----> 3  `animal.quack()`

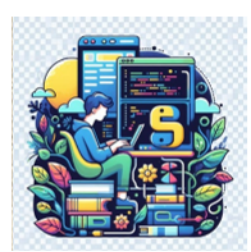
**AttributeError:** 'Cat' object has no attribute 'quack'



# PROGRAMMING WITH PYTHON

## Duck Typing

- One of the most used examples of duck typing is an iteration in python.
  - How can a for loop be written with every iterable in python?
  - What makes an object iterable?
- Duck typing comes into play here.
- No matter how different these objects are in terms of application, they are treated equally because of duck typing.



# PROGRAMMING WITH PYTHON

## Recap

- Polymorphism
  - Operator Polymorphism
  - Function Polymorphism
  - Class Polymorphism
- Polymorphism and Inheritance
  - Method Overriding
  - Duck Typing



**THANK YOU**

---

**Lekha A**  
Computer Applications